Operating Systems Lab – Assessment 10

Shortest Remaining Time First (SRTF) Scheduling Algorithm

Upayan Mazumder - 24BDS0367

Problem Statement:
Create a LINUX C program to implement the Shortest Remaining Time First (SRTF) Scheduling Algorithm. Processes will be scheduled based on their arrival and burst time. Print the average turnaround time and average waiting time.

Given Process Data:

Process	Burst Time	Arrival Time
P1	6	2
P2	2	5
P3	8	1
P4	3	0
P5	4	4

Source Code (`main.c`):

```c
#include <stdio.h>
#include <limits.h>

struct Process {
    int pid;
    int at;
    int bt;
    int rt;
    int ct;
    int tat;
    int wt;
};

int main(void) {
    int n;
    printf("Registration Number: 24BDS0367\n");
    printf("Name: Upayan Mazumder\n");
    printf("Operating Systems Lab Assessment 10\n\n");

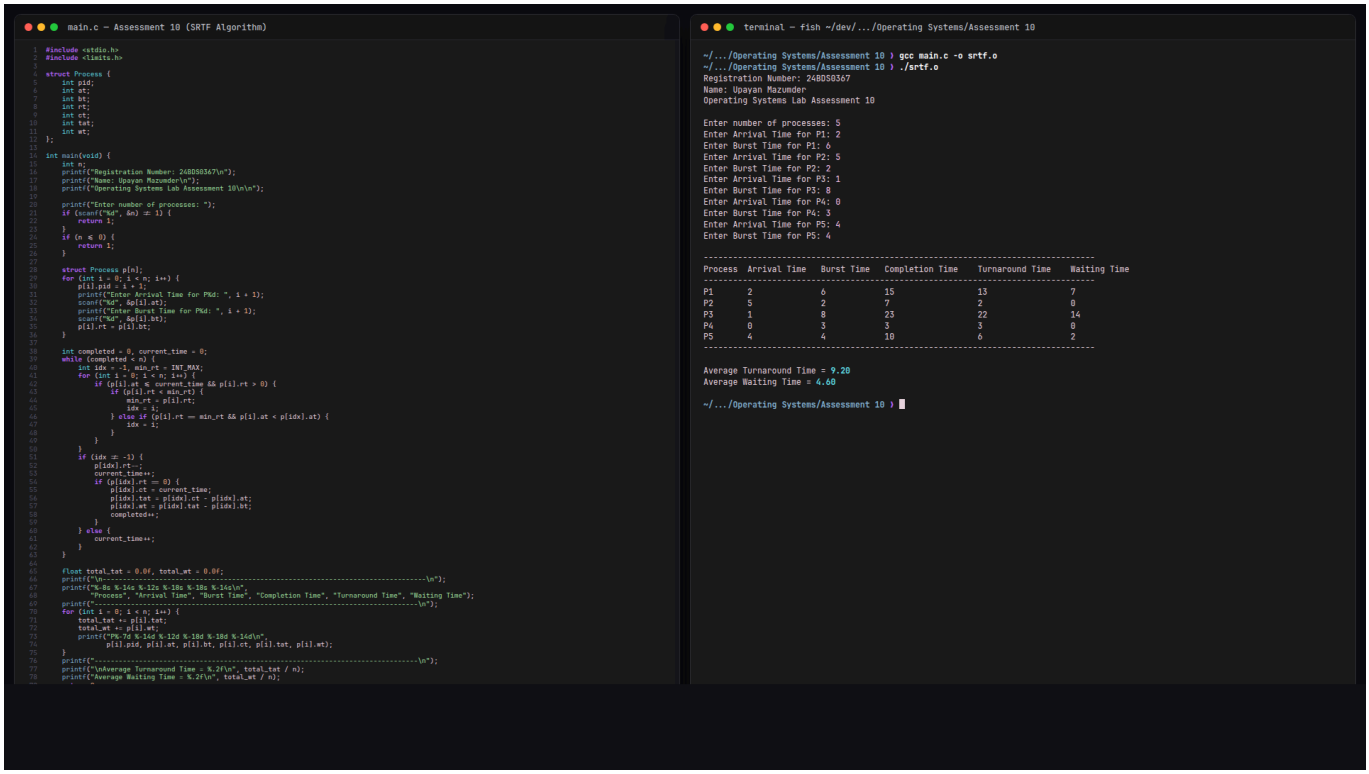
    printf("Enter number of processes: ");
    if (scanf("%d", &n) != 1) {
        return 1;
    }
    if (n <= 0) {
        return 1;
    }

    struct Process p[n];
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("Enter Arrival Time for P%d: ", i + 1);
        scanf("%d", &p[i].at);
        printf("Enter Burst Time for P%d: ", i + 1);
        scanf("%d", &p[i].bt);
        p[i].rt = p[i].bt;
    }

    int completed = 0, current_time = 0;
    while (completed < n) {
        int idx = -1, min_rt = INT_MAX;
        for (int i = 0; i < n; i++) {
            if (p[i].at <= current_time && p[i].rt > 0) {
                if (p[i].rt < min_rt) {
                    min_rt = p[i].rt;
                    idx = i;
                } else if (p[i].rt == min_rt && p[i].at < p[idx].at) {
                    idx = i;
                }
            }
        }
        if (idx != -1) {
            p[idx].rt--;
            current_time++;
            if (p[idx].rt == 0) {
                p[idx].ct = current_time;
                p[idx].tat = p[idx].ct - p[idx].at;
                p[idx].wt = p[idx].tat - p[idx].bt;
                completed++;
            }
        } else {
            current_time++;
        }
    }

    float total_tat = 0.0f, total_wt = 0.0f;
    printf("\n--------------------------------------------------------------------------------\n");
    printf("%-8s %-14s %-12s %-18s %-18s %-14s\n",
           "Process", "Arrival Time", "Burst Time", "Completion Time", "Turnaround Time", "Waiting Time");
    printf("--------------------------------------------------------------------------------\n");
    for (int i = 0; i < n; i++) {
        total_tat += p[i].tat;
        total_wt += p[i].wt;
        printf("P%-7d %-14d %-12d %-18d %-18d %-14d\n",
               p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
    }
    printf("--------------------------------------------------------------------------------\n");
    printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);
    printf("Average Waiting Time = %.2f\n", total_wt / n);
    return 0;
}
```

Screenshot of Code and Output



Output Metrics Summary:

Process	Arrival Time	Burst Time	Completion Time	Turnaround Time	Waiting Time
P1	2	6	15	13	7
P2	5	2	7	2	0
P3	1	8	23	22	14
P4	0	3	3	3	0
P5	4	4	10	6	2

- **Total Turnaround Time:** $13 + 2 + 22 + 3 + 6 = 46$
- **Average Turnaround Time:** $\frac{46}{5} = 9.20$
- **Total Waiting Time:** $7 + 0 + 14 + 0 + 2 = 23$
- **Average Waiting Time:** $\frac{23}{5} = 4.60$